

Contents

Test Strategy	1
1. Why this document exists	1
2. The three principles	1
2.1 Tests are the contract	1
2.2 Refactor under a regression net or not at all	1
2.3 The pyramid scales by frequency, not by ceremony	2
3. The test pyramid for this system	2
4. Layer 1 — Unit / per-rule / per-form / per-config	2
5. Layer 2 — Integration / per-API / per-datalist	3
6. Layer 3 — End-to-end / lifecycle	3
7. Layer 4 — Performance	4
8. Layer 5 — Equivalence (pre-/post-refactor)	5
9. The regression net for ADR-031, slice by slice	5
10. The continuous-execution strategy	6
10.1 A make test (or equivalent) entry point	6
10.2 A pre-push git hook	6
10.3 A pre-deploy gate	6
11. Failure response	6
12. What this strategy does NOT cover	6
13. The first concrete step	7
14. Maintenance	7

Test Strategy

Status	Proposed (2026-05-07)
Owner	Farmers Portal architecture team
Audience	Engineering, MAFSN ICT, customer architecture lead
Companion	ADR-031 (unified rule engine)

1. Why this document exists

The system is pre-production. The customer’s directive is “*whatever we do, the working things stay working as is*”. This document lays out the regression-test architecture that delivers on that directive — both for the unified-rule-engine refactor coming in ADR-031 and for every future change.

It is not a list of tests to write once. It is the **architecture** of how testing fits into the project’s lifecycle: which tests run when, what each layer guarantees, what failures mean, and how the suite evolves as the system grows.

2. The three principles

2.1 Tests are the contract

A behaviour is “supported” if and only if a test asserts it. Anything else is incidental and can drift. When a customer says “voucher redemption produces an EXPENSE event”, the proof of that behaviour is the test asserting it — not the prose in the SAD, not the operator manual, not someone’s memory.

2.2 Refactor under a regression net or not at all

Before any non-trivial refactor (the rule-engine unification, a major form schema change, a Joget upgrade), the regression net must already cover the behaviour being refactored. If the test isn’t there, the refactor is not safe — it’s just a guess. The first task of any refactor is therefore: *write the test that proves the current behaviour, run it green, then start changing things*.

2.3 The pyramid scales by frequency, not by ceremony

Fast tests run on every change. Slow tests run on every commit. End-to-end tests run before every deploy. Heavyweight equivalence tests run before/after major refactors. Continuous *frequency* matters more than continuous *integration ceremony* — at single-team scale, what matters is that the right test runs at the right moment, not that there's a Jenkins dashboard.

3. The test pyramid for this system

Equivalence tests (rule × fixture)	pre-/post-refactor only
Performance tests (perf baseline + load)	pre-deploy
End-to-end / lifecycle (e2e_im, e2e_subsidy, L4)	nightly + pre-deploy
Integration / per-API / per-DL (API smoke, datalist render)	per-commit
Unit / per-rule / per-form / per-config (test_rules, test_forms, test_seeds)	per-change

Five layers, each with a clear purpose, scope, and run frequency.

4. Layer 1 — Unit / per-rule / per-form / per-config

Purpose: assert the smallest unit of system behaviour — one rule, one form definition, one master-data seed.

Scope today (existing):

- Java unit tests in `gs-plugins/*/src/test/java/` for the SQL-path / fast-path evaluators.
- Schema validity of every form JSON in `_forms/` — the form-creator-api rejects invalid JSON on push.

Scope to add:

Test	What it covers	Authoring location
<code>test_rules_per_rule.py</code>	For each <code>mm_rule</code> row: feed a fixture record where the rule should fire → assert it fires; feed a record where it shouldn't → assert no issue.	<code>_tooling/test_rules_per_rule.py</code>
<code>test_forms_save_load.py</code>	For each form: seed a row, read it back, assert all fields round-trip. Catches Hibernate mapping drift.	<code>_tooling/test_forms_save_load.py</code>
<code>test_md_seeds.py</code>	Confirm every MD lookup table has 1 row; no broken FK references; no orphan codes.	<code>_tooling/test_md_seeds.py</code>

Test	What it covers	Authoring location
<code>test_userview_menus.py</code>	Every CrudMenu's <code>editFormId</code> , <code>addFormId</code> , <code>datalistId</code> resolves to a live form/datalist. (CLAUDE.md “Userview menu hygiene”.)	<code>_tooling/test_userview_menus.py</code>

Run frequency: per-change. The author runs these locally before pushing.

What “pass” means: the assertion holds. Stable across repeated runs (no flakes). <30 seconds total to run.

What failure means: the change broke a unit-level invariant. Fix the change before merging.

5. Layer 2 — Integration / per-API / per-datalist

Purpose: assert that one component talks correctly to its immediate neighbours — an API endpoint to its handler, a datalist to its binder.

Scope today (existing):

- `_tooling/test_im_e2e.py` runs a 5-report smoke pass that confirms every IM report’s SQL parses and returns rows.
- `_tooling/run_l4_scenarios.py` runs eligibility scenarios via the citizen API end-to-end.

Scope to add:

Test	What it covers
<code>test_api_smoke.py</code>	Every published REST endpoint (<code>regbb/eval</code> , <code>regbb/submit</code> , <code>regbb/mdm/</code> , <i>budget/</i> , <code>formcreator/*</code>) returns 200 for a valid input or a documented 4xx for an invalid input.
<code>test_datalist_render.py</code>	Every datalist in <code>_datalists/</code> renders without error against the live database — page 1, page 2, with each filter populated.
<code>test_userview_render.py</code>	Every menu in the userview opens without throwing “plugin not installed” or “data list not exist” errors.

Run frequency: per-commit. CI-style, although today this is run manually from the command line.

What “pass” means: every endpoint and every render works end-to-end. <5 minutes total.

What failure means: an integration boundary broke. Could be a column rename, a missing seed row, a plugin not registered. Bisect to find the change.

6. Layer 3 — End-to-end / lifecycle

Purpose: assert that a complete user journey from start to finish produces the expected outcome.

Scope today (existing):

- `_tooling/test_im_e2e.py` — citizen submits → eligibility worker → auto-approve → voucher issue → counter redeem → distribution receipt. ~15 seconds.
- `_tooling/test_im_stacking.py` — multi-programme stacking against one applicant. Confirms a single applicant can hold multiple concurrent vouchers.

- `_tooling/run_l4_scenarios.py` — 20 eligibility scenarios across 4 programmes. The eligibility regression gate.
- `_tooling/test_budget_suite.py` — budget reports + maker-checker controls + threshold alerts.

Scope to add (rule-engine unification specifically):

Test	What it covers
<code>test_l5_quality_regression.py</code>	Each of the 21 quality rules: trigger a save on the relevant form with the issue present → assert QualityBanner state turns red/amber → assert gate behaviour blocks status transition → fix the issue → re-save → assert banner clears.
<code>test_voucher_lifecycle.py</code>	Issue → partially redeem → partially redeem → expire (sweeper). Confirms RELEASE_COMMITMENT for the unspent portion.
<code>test_voucher_cancel.py</code>	Issue → cancel → confirm RELEASE_COMMITMENT for the full amount.
<code>test_application_decision_paths.py</code>	Submit → auto-approve / pending review / sent back / rejected paths each produce the right downstream behaviour (voucher / no voucher / clarification request).

Run frequency: nightly + pre-deploy.

What “pass” means: a real user journey works end-to-end. The eligibility test takes ~30 seconds; the longest lifecycle test ~60 seconds. <10 minutes total.

What failure means: a feature regressed at the user level. This is the test layer the customer experiences when something breaks.

7. Layer 4 — Performance

Purpose: assert that the system stays within its performance budget under realistic load.

Scope today (existing):

- `_tooling/load_test.py` — two modes: read-side report query (10 reports × 30 hits each, target <850 ms p95) and HTTP round-trip via `/budget/timeseries`.
- `_07_Implementation_Guides/perf_baseline.md` — captured baseline numbers.

Scope to add:

Test	What it covers
Per-save evaluation latency	Save a record on every QualityBannered form; assert the post-processor’s evaluation budget stays <500 ms. Pre-/post-unification regression check for ADR-031.
Citizen-wizard render	Open the citizen application wizard 50 times concurrently (simulating 50 farmers at programme launch). Assert p95 render time <2 s.
Bulk operator action	Approve 50 applications in one bulk-action transaction. Assert all 50 voucher rows materialise within 60 seconds.

Run frequency: pre-deploy. Documented baseline updated quarterly.

What “pass” means: budgets hold. p95 numbers stable across runs.

What failure means: something got slower. Bisect to the change; profile the bottleneck.

8. Layer 5 — Equivalence (pre-/post-refactor)

Purpose: assert that two implementations of the same logic produce identical outputs over a representative input corpus. The unique tool for safe refactors.

Scope today: none. Created for ADR-031.

Scope to add for ADR-031:

Test	What it covers
<code>test_rules_engine_equivalence.py</code>	For each rule × each fixture record, run the legacy engine (form-quality-runtime + eligibility evaluator-as-is) and the unified engine (post-merge). Assert byte-for-byte outcome equivalence: same disposition, same issue list, same gate behaviour, same audit row contents.

The test runs against a frozen corpus of test fixtures (the existing dev fixture + the L4 scenarios). Both engines must be runnable in parallel during the dual-write window of ADR-031 Slice C — that is the engineering precondition for using equivalence tests at all.

Run frequency: pre-/post-each slice of the ADR-031 migration. Once unification is complete and the legacy engine is retired (Slice E), this test is **archived** — its job is done. Equivalence tests are not permanent infrastructure; they are migration tooling.

What “pass” means: the unified engine reproduces the legacy engine’s behaviour exactly.

What failure means: the unification has introduced a behavioural change. Either the unification is wrong (fix the engine) or the legacy behaviour was a bug (document and proceed). No silent divergence.

9. The regression net for ADR-031, slice by slice

Mapping each slice of the ADR-031 migration to the test layer that gates its acceptance:

Slice	What changes	Tests gating acceptance
A — Schema unification	New columns on <code>mm_determinant</code> (now <code>mm_rule</code>); no behavioural change	Layer 1 (forms round-trip), Layer 3 (L4 passes)
B — Data migration <code>qa_rule</code> → <code>mm_rule</code>	New rows; legacy still authoritative	Layer 5 (equivalence test passes for both stores)
C — Switch read paths	Form-quality-runtime reads from <code>mm_rule</code> ; QualityBanner reads from <code>rule_outcome</code>	Layer 5 (equivalence test passes), Layer 3 (L4 passes, new L5 passes), Layer 4 (per-save latency budget holds)
D — Unified authoring UI	New CRUD list with <code>scope</code> filter	Layer 2 (datalist + form render), Layer 1 (every existing rule remains editable)
E — Retire formQuality	Delete the formQuality app	Layer 1 (no orphan references), Layer 2 (no broken menu), Layer 3 (L4 + L5 still pass)

The L4 parity test runs at the end of every slice. If L4 regresses, the slice is rolled back. This is the canary.

10. The continuous-execution strategy

Today the project doesn't have CI. Tests run when the engineer remembers to run them. That's been adequate at single-team scale; at pre-production with the rule-engine refactor coming, it isn't anymore. Three concrete steps:

10.1 A `make test` (or equivalent) entry point

A single shell command that runs Layers 1 + 2 + 3 in sequence, prints a per-test pass/fail summary, exits non-zero on any failure. ~10 minutes total. Run before any push. Suggested:

```
make test          # layers 1-3, ~10 min
make test-perf     # layer 4, ~5 min, run pre-deploy
make test-equiv    # layer 5, ad-hoc, only during refactors
```

`make test` is just a thin wrapper around the existing `_tooling/test_*.py` scripts plus the new ones. No infrastructure required; just a Makefile.

10.2 A pre-push git hook

`.git/hooks/pre-push` runs `make test` and refuses the push if anything fails. Trivial to install; saves the engineer from a broken main branch.

10.3 A pre-deploy gate

Before any JAR upload to the dev or production Joget instance, a checklist:

- ☐ `make test` passed within the last hour.
- ☐ `make test-perf` baseline within tolerance (no >50 % regression on any p95).
- ☐ L4 parity test passed (this is in `make test`, but call it out separately because it's the eligibility canary).
- ☐ Affected slice's specific test (e.g. `test_15_quality_regression.py` for ADR-031 Slice C) passed.

This is the “we deploy because it's safe, not because we hope” gate.

11. Failure response

When a test fails, the response is graded by which layer failed:

- **Layer 1 fails** — the change is broken at unit level. Don't push. Fix locally.
 - **Layer 2 fails** — an integration boundary broke. Bisect; could be a column rename or a missing seed.
 - **Layer 3 fails** — a user journey regressed. **This is the most expensive class of failure** because the test usually doesn't pinpoint the cause; you'll trace through the lifecycle to find it. Worth investing extra time to make Layer 3 failures produce useful logs.
 - **Layer 4 fails** — performance got worse. Profile; usually a missing index or a quadratic loop newly introduced.
 - **Layer 5 fails** (during a refactor) — the refactor changes behaviour. Either the change is wrong or the legacy was buggy. Either way, **stop the refactor** and resolve.
-

12. What this strategy does NOT cover

To set the boundary explicitly:

- **Browser-based UI testing.** Selenium / Playwright / similar. Not a 2026 task; the operator surfaces are stable enough that prose-based UAT (the existing `_uat_guide` document) catches what unit + e2e tests don't.
- **Security testing.** Pen testing, dependency-vulnerability scanning, OWASP top-ten regression. The MinAgri IT TO-DO captures these as customer-side responsibilities.

- **Disaster-recovery testing.** Backup + restore drills. Covered in `_07_Implementation_Guides/backup_restore_runbook`.
- **Localisation testing.** Sesotho translation accuracy. Out of scope for the regression net; handled by MAFSN's bilingual reviewer.
- **Accessibility testing.** WCAG conformance, screen-reader compatibility. A separate workstream if MAFSN procures it.

These are valid concerns; they are simply not addressed here. Each has its own document or its own owner.

13. The first concrete step

Before ADR-031's first slice begins, **author the equivalence test** (`test_rules_engine_equivalence.py`). Run it green against the legacy engine alone (it should produce the same outcome twice in a row). That confirms the test infrastructure works. Only then begin Slice A.

This is the embodiment of principle §2.2: refactor under a regression net or not at all.

14. Maintenance

This document is updated whenever:

- A new test is added → its row goes into the appropriate layer's table.
- A new layer is added → revisit the pyramid section.
- A test is retired → mark it archived (don't delete the entry; future readers should know the test existed).
- A failure response changes → update §11.

Last review date in the header. Next scheduled review: at the conclusion of ADR-031's migration.